

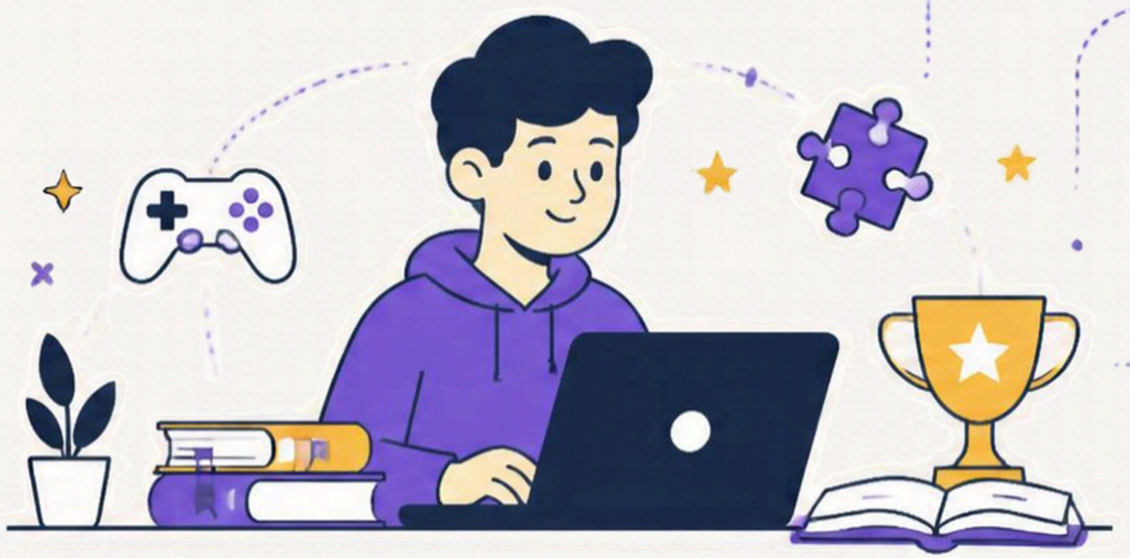
PROJETOS E CONFIGURAÇÃO DE SISTEMAS

APRENDER BRINCANDO TRANSFORMA O FUTURO.

LUCAS ALVES BITTENCOURT · JOÃO VICTOR · PATRICIA SALES MANSANO

APRENDER COM JOGOS PODE SER MAIS DO QUE UMA PARTIDA

Como tornar a aprendizagem com jogos mais contínua, acessível e envolvente?



CONTEÚDO E DESCOBERTA

Encontrar atividades adequadas ao nível, disciplina e tema pode exigir navegar por conteúdos dispersos.

POUCO ACOMPANHAMENTO

Sem histórico e desempenho, o aluno tem pouca visibilidade sobre sua evolução.

ENGAJAMENTO ALÉM DA PARTIDA

Continuidade, identidade e interação podem ampliar a motivação para aprender e jogar novamente.

A SOLUÇÃO

LÚDICA CONECTA JOGOS, APRENDIZAGEM E INTERAÇÃO



Uma plataforma web de aprendizagem baseada em jogos, pensada para dar continuidade à experiência educacional.

JOGOS E CONTEÚDO

Atividades educacionais em diferentes formatos, organizadas por nível, disciplina e tema.

EXPERIÊNCIA CONTÍNUA

Histórico e desempenho ajudam o aluno a acompanhar sua evolução e retomar atividades.

DIFERENTES PERFIS

Aluno, professor e administrador possuem papéis e experiências próprias na plataforma.

IDENTIDADE E INTERAÇÃO

Gamertag, avatar, socialização e ranking

DESCOBRIR → JOGAR → ACOMPANHAR → CONTINUAR

O PONTO DE PARTIDA

PROJETO EXISTENTE

O Lúdica já possui uma primeira versão funcional e uma proposta de evolução definida pelo grupo anterior no semestre 2026/1 da disciplina.

JÁ IMPLEMENTADO

- Login e Home do usuário
- Cadastro e Perfil
- Criação e personalização do Avatar
- Solicitação e aceitação de amizades
- Biblioteca para navegação entre jogos
- Detalhamento dos jogos e histórico de partidas
- Visualização parcial das pontuações dos amigos

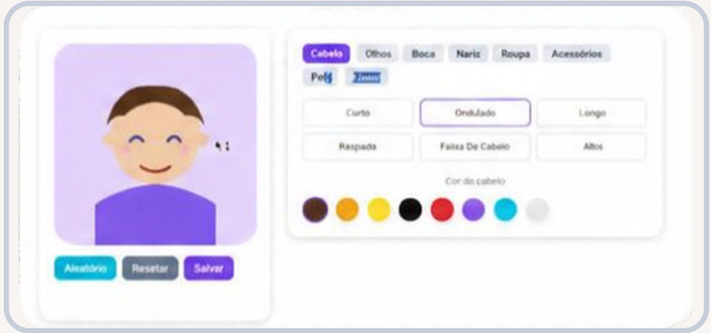
PROPOSTA INICIAL PARA EVOLUÇÃO

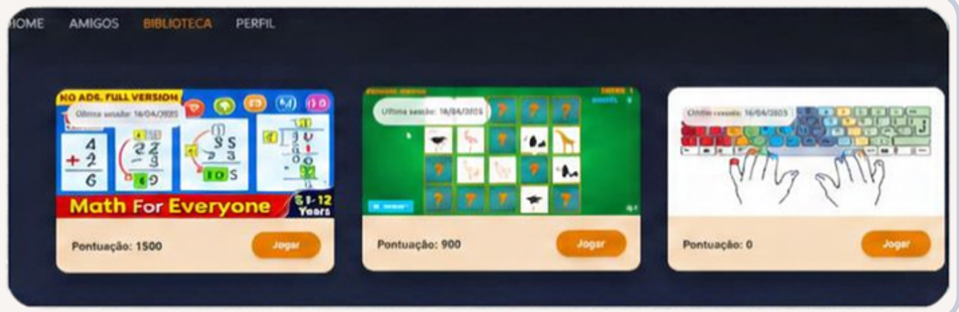
Trabalho futuro definido em 2026.1

- Integração dos jogos como aplicações separadas via API **(Jogo palavra Cruzada digital)**
- Visualização do perfil dos amigos, incluindo avatar e pontuações
- Loja de itens para personalização do Avatar utilizando pontos
- Sistema de conquistas

Versão recebida pelo grupo — 2026.1







PROJETO HERDADO → BASE FUNCIONAL + TRABALHO FUTURO → INÍCIO DA NOSSA AVALIAÇÃO

Nosso primeiro passo foi executar, testar e compreender essa base antes de propor novas evoluções.

ENTENDER A BASE ANTES DE EVOLUIR

Estudamos a documentação existente, configuramos o ambiente e percorremos os principais fluxos da aplicação para compreender seu funcionamento.

DOCUMENTAÇÃO → AMBIENTE → EXECUÇÃO → TESTES

PRINCIPAIS PROBLEMAS

IDENTIDADE DO USUÁRIO

alice = ALICE

Gamertags equivalentes pela diferença entre maiúsculas e minúsculas podiam ser tratadas como identificadores distintos.

HISTÓRICO E RANKING

Usuário fixo no código

Histórico e ranking utilizavam id_usuario = 1, em vez do usuário autenticado.

HOME

Conteúdo ainda estático

Parte da página inicial permanecia mockada e não refletia os dados reais da aplicação. Sem botão de voltar para a biblioteca

PERFIS E ACESSO

Administrador sem experiência específica

O login funcionava, mas ainda não existia interface ou comportamento próprio para esse perfil.

BASE TECNOLÓGICA Docker · Node.js React + JavaScript · HTML5 / CSS3 · PostgreSQL

Do diagnóstico à evolução do projeto →

NOSSA EVOLUÇÃO

SPRINT EM NÚMEROS

O trabalho avançou da compreensão da base recebida para a correção de funcionalidades, evolução da experiência e integração da aplicação.

01



ENTENDER E DIAGNOSTICAR

- **12** testes funcionais
- **8** inconsistências mapeadas
- **2** correções já validadas

Executamos a base recebida, testamos os principais fluxos e definimos prioridades.

02



CORRIGIR E VALIDAR

- **7** atividades planejadas
- **11** validações executadas
- **11/11** validações aprovadas

Transformamos os achados iniciais em correções verificadas por testes automatizados e validação visual.

03



INTEGRAR E CONSOLIDAR

- **5** atividades priorizadas
- **3** concluídas
- **1** levantamento realizado
- **1** direcionada à próxima Sprint

A Sprint 3 avançou na integração da aplicação e na consolidação das evoluções realizadas.

Da evolução à entrega →

O QUE EVOLUÍMOS E VAMOS DEMONSTRAR

As entregas das últimas Sprints consolidaram funcionalidades existentes e prepararam a aplicação para uma experiência mais integrada.

SPRINT 2
FUNCIONALIDADES CONSOLIDADAS

Gamertag única
Bloqueio no cadastro e na edição, sem diferenciar maíusculas e minúsculas.

Perfil persistido
Alterações vinculadas ao próprio usuário autenticado.

Avatar no login
Avatar recuperado após a autenticação.

Tipo de jogador
Cadastro como Aluno ou Professor.

SPRINT 3
EVOLUÇÃO E INTEGRAÇÃO

Docker integrado
Front-end, back-end e banco executados de forma integrada.

Histórico e ranking
Dados associados ao usuário autenticado.

Detalhe do jogo
Tela aprimorada com desempenho, histórico e ranking.

Home renovada
Nova experiência de navegação e conteúdo.

Login por e-mail
Nova opção de autenticação.

Proteção de Perfil e Avatar
Alterações protegidas pelo usuário autenticado.

Administrador
levantamento realizado

Acessar o perfil dos amigos
próxima Sprint

EVOLUÇÕES FUNCIONANDO NA APLICAÇÃO

DEMONSTRAÇÃO

ROTEIRO DA APRESENTAÇÃO

1

SUBIR TUDO

DOCKER COMPOSE UP --BUILD E O SEED

2

ENTRAR COM E-MAIL

ALICE@EMAIL.COM E AVATAR CARREGADO

3

HOME

BUSCA, MAIS POPULARES E CONTINUE JOGANDO

4

DETALHE DO JOGO

DESEMPENHO, HISTÓRICO E RANKING DOS AMIGOS

5

PERFIL

EDITAR DADOS E TENTAR UMA GAMERTAG REPETIDA

6

OUTROS PERFIS

PROF_ANA E ADMIN NO MESMO JOGO

ANTES DA DEMONSTRAÇÃO

Inicialização do ambiente

Etapas de preparação executadas no terminal antes de iniciar a gravação.

Docker Compose

```
docker compose up --build
```

Seed do banco

```
docker compose exec backend npm run seed
```

Backend

```
http://localhost:3000
```

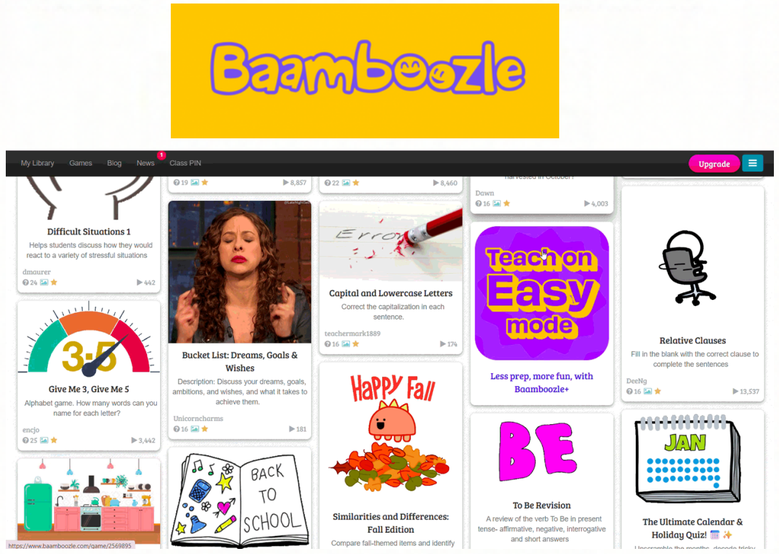
Front-end

```
http://localhost:5173
```

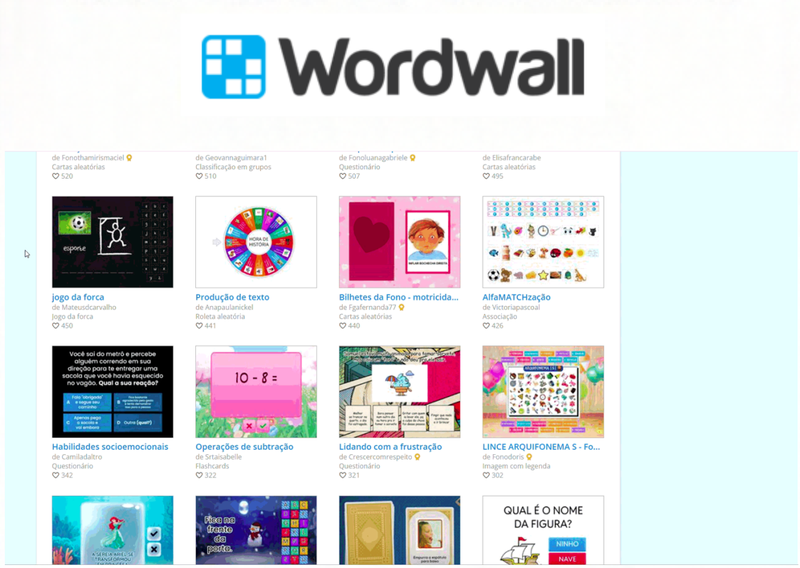
3

O teste começa em instantes

APLICAÇÕES SIMILARES EM ATIVIDADE



Simplicidade de navegação e acesso às atividades



Experiência educacional baseada em jogos



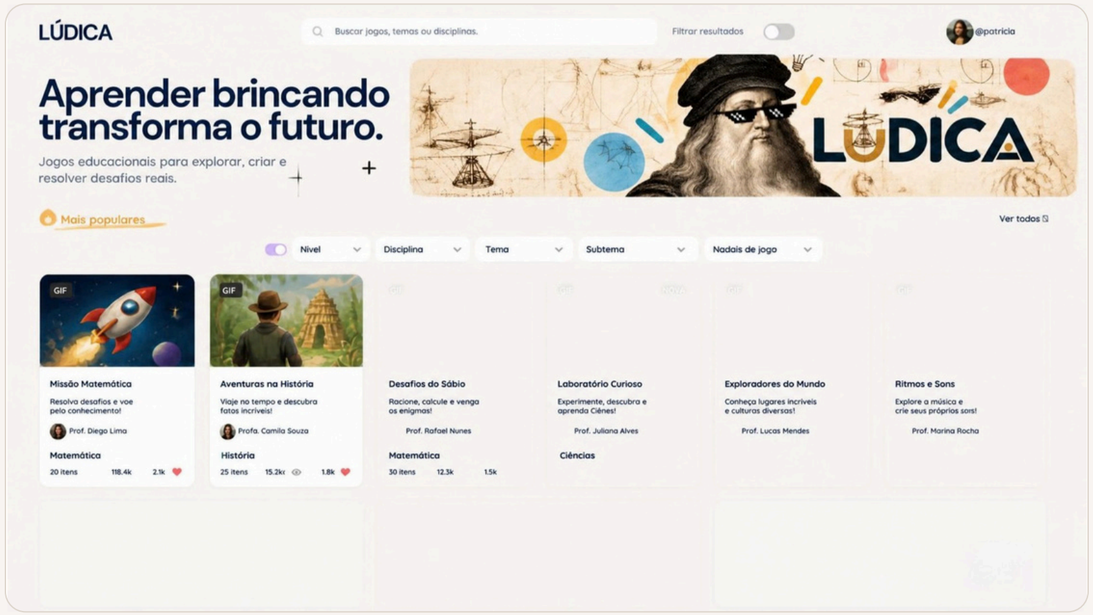
Biblioteca de jogos e experiência do aluno

REFERÊNCIAS → PROTÓTIPO → IMPLEMENTAÇÃO

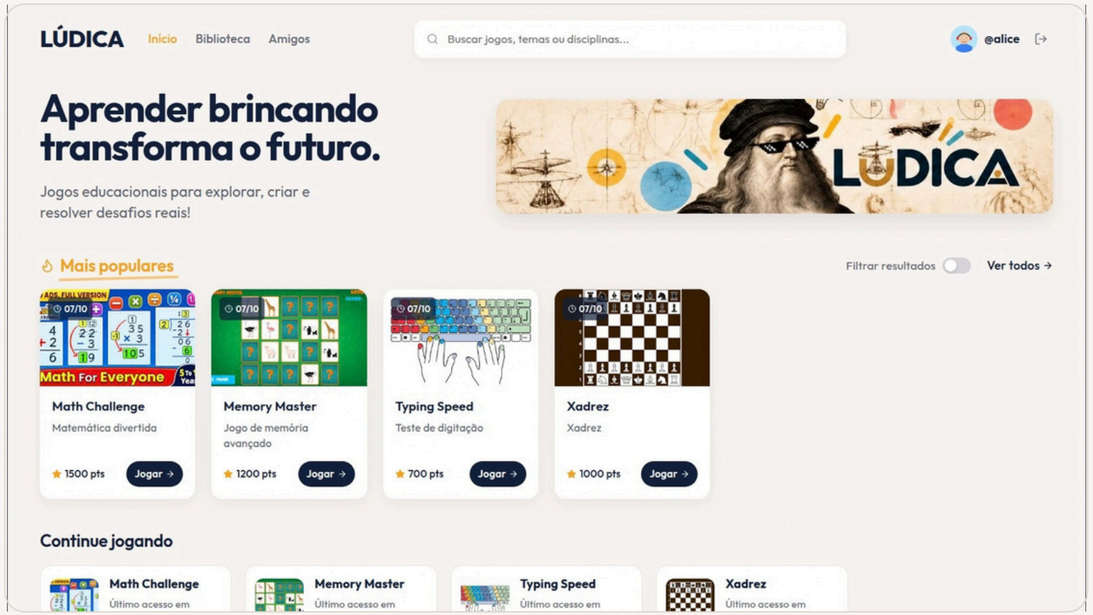
INTERFACE

DO PROTÓTIPO À TELA

PROTÓTIPO



IMPLEMENTADO NESTA SPRINT

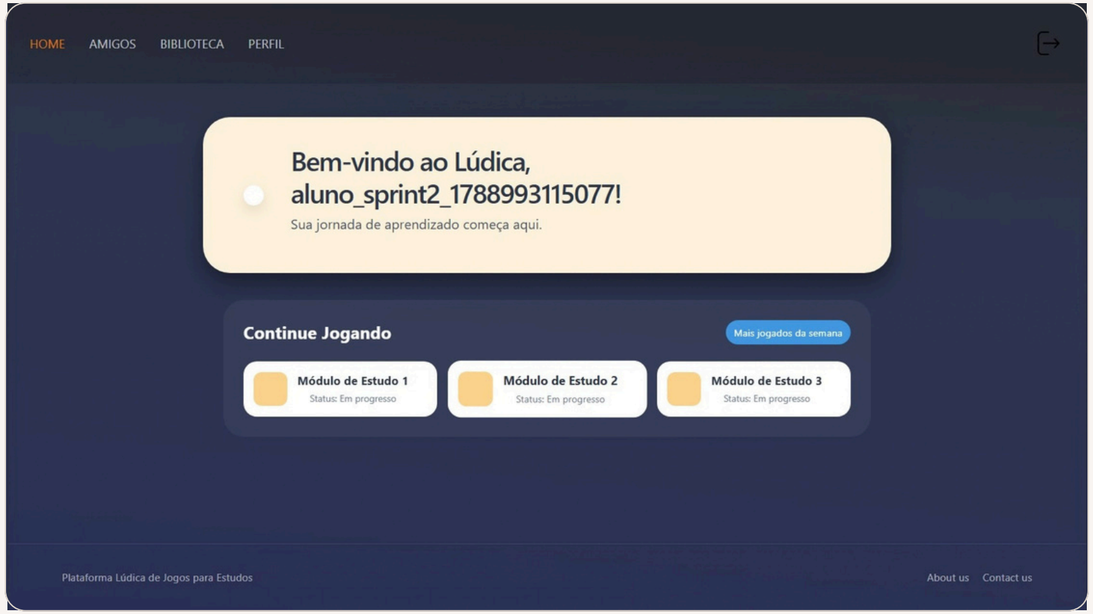


MESMA PALETA, TIPOGRAFIA (OUTFIT), HIERARQUIA E ORGANIZAÇÃO EM CARDS.

ANTES E DEPOIS

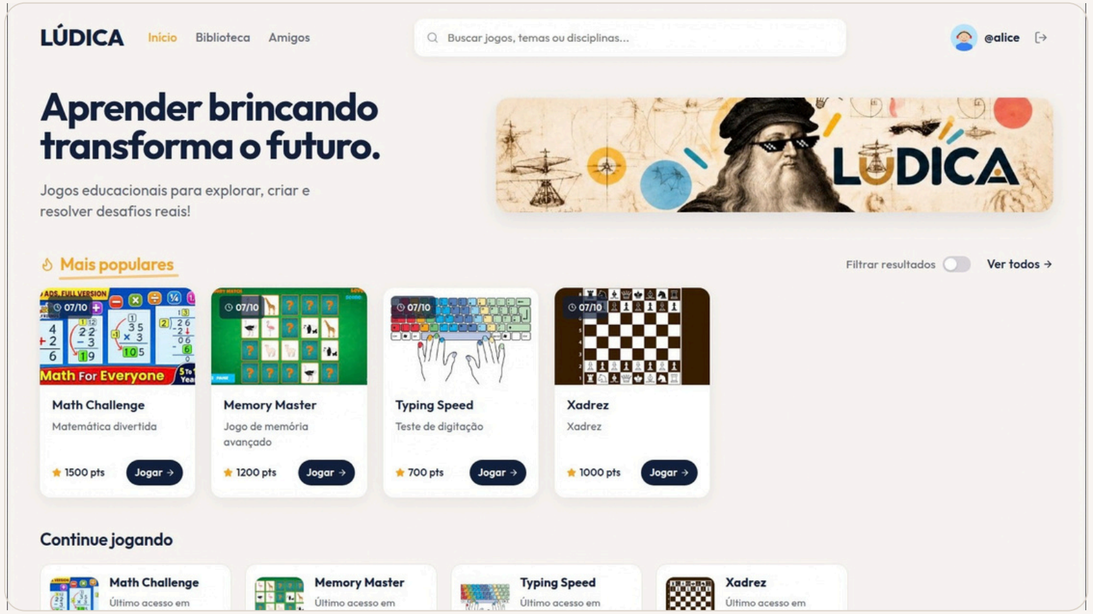
PÁGINA INICIAL

ANTES · SPRINT 2



CARDS FIXOS ("MÓDULO DE ESTUDO 1, 2, 3") E SÓ O MENU NO CABEÇALHO.

DEPOIS · SPRINT 3



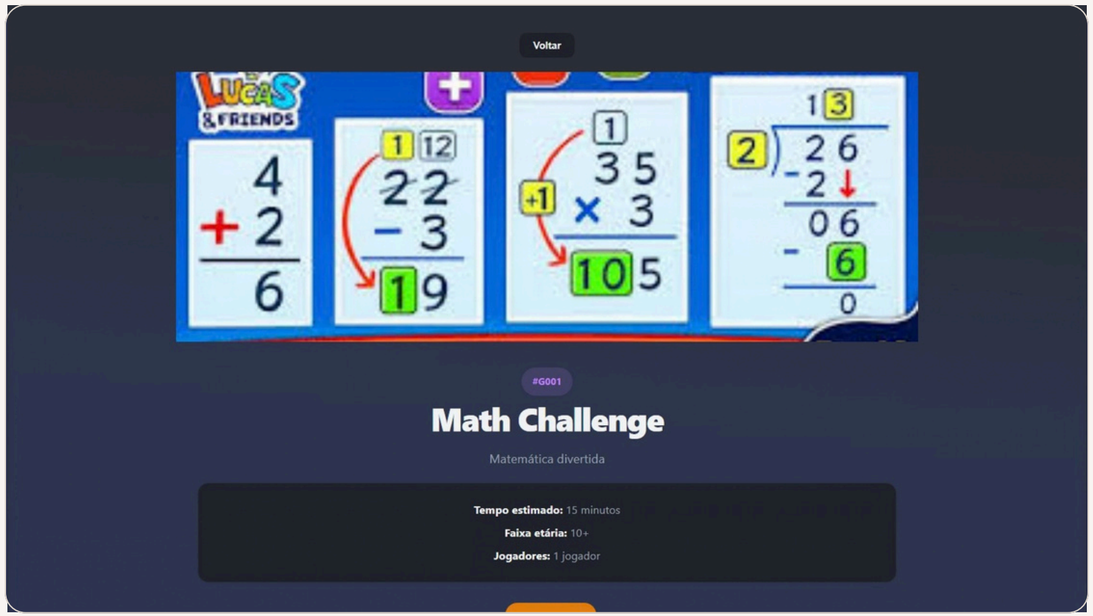
JOGOS VINDOS DO BANCO, BUSCA GLOBAL, AVATAR E GAMERTAG NO CABEÇALHO, "CONTINUE JOGANDO" PELO ÚLTIMO ACESSO.

Home.jsx · Navbar.jsx · GameCard.jsx · Layout.jsx

ANTES E DEPOIS

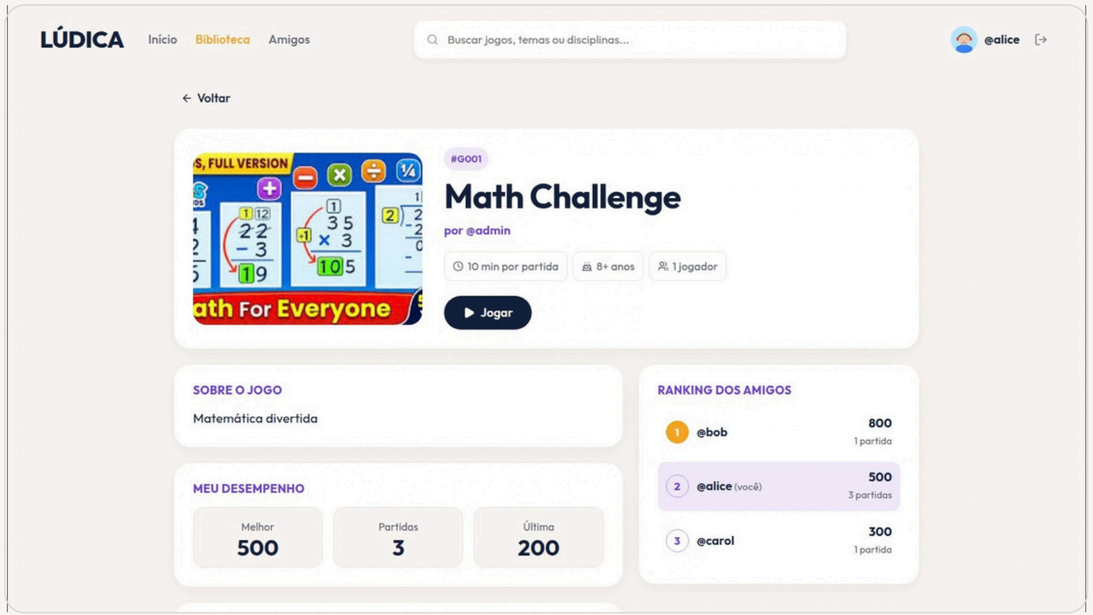
DETALHE DO JOGO

ANTES · SPRINT 2



TUDO EMPILHADO EM UMA COLUNA; PARA VER O RANKING ERA PRECISO ROLAR A PÁGINA INTEIRA.

DEPOIS · SPRINT 3



VOLTAR, AUTOR, TEMPO, FAIXA ETÁRIA E JOGADORES NO TOPO; DESEMPENHO E RANKING VISÍVEIS JUNTOS.

GameDetails.jsx · gamesdetails.module.css · gamesService.js

NO CÓDIGO · GAMEDETAILS.JSX

MENOS ESPERA, IMAGEM CERTA, SEM ZEROS

ANTES

```
// uma chamada depois da outra
const gameData = await getGameById(id);
const historyData =
  await getHistoryByUserAndGame(...);
const rankingData =
  await getRankingByGame(...);

// caminho fixo pelo id do jogo
<img src={`/images/${game?.id_jogo}.jpeg`} />

// sem partidas, mostrava 0
history.length > 0 ? Math.max(...) : 0
```

DEPOIS

```
// as três ao mesmo tempo
const [gameData, historyData, rankingData] =
  await Promise.all([
    getGameById(id),
    podeJogar ? getHistory... : [],
    isAluno ? getRankingByGame... : [],
  ]);

// imagem cadastrada no banco
<img src={game.url_imagem} alt="" />

// sem partidas, convida a jogar
"Você ainda não jogou. Que tal..."
```

INFRAESTRUTURA · ATIVIDADE 1

A APLICAÇÃO INTEIRA NO DOCKER

ANTES · TRÊS PASSOS MANUAIS

BANCO
NO DOCKER

BACK-END
NPM INSTALL E NPM RUN DEV, TERMINAL 1

FRONT-END
NPM INSTALL E NPM RUN DEV, TERMINAL 2

DEPOIS · UM COMANDO

```
docker compose up --build
```



DB
POSTGRES · 5433

BACKEND
NODE 22 · 3000

FRONTEND
NGINX · 5173

O BACK-END SÓ INICIA QUANDO O BANCO RESPONDE (HEALTHCHECK).

NO CÓDIGO · DOCKER

O BACK-END ACHA O BANCO PELO NOME

DB.JS · ANTES

```
new Pool({
  host: 'localhost',
  port: 5433,
```

DB.JS · DEPOIS

```
new Pool({
  host: process.env.DB_HOST || 'localhost',
  port: Number(process.env.DB_PORT) || 5433,
```

DENTRO DO DOCKER VIRA DB:5432; FORA DELE, O MESMO DE SEMPRE.

DOCKER-COMPOSE.YML · SERVIÇOS NOVOS

```
backend:
  build: ../back-end
  environment:
    - DB_HOST=db
    - DB_PORT=5432
  ports:
    - "3000:3000"
  depends_on:
    db:
      condition: service_healthy
frontend:
  build: ../front-end
  ports:
    - "5173:80"
```

Novos: back-end/Dockerfile · front-end/Dockerfile · nginx.conf · .dockerignore

AUTENTICAÇÃO · USERSERVICE.JS

LOGIN POR GAMERTAG OU E-MAIL

ANTES

```
SELECT * FROM usuario
WHERE nome_usuario = $1
```

DEPOIS

```
SELECT * FROM usuario
WHERE LOWER(nome_usuario) = LOWER($1)
   OR LOWER(email) = LOWER($1)
```

NO LOGIN.JSX, O CAMPO PASSOU A DIZER "GAMERTAG OU E-MAIL".

RESPOSTA DA API PARA CADA ENTRADA

ENVIADO	RESPOSTA
ALICE	200 · TOKEN
ALICE@EMAIL.COM	200 · TOKEN
E-MAIL COM ESPAÇOS	200 · TOKEN (TRIM)
SENHA ERRADA	401 · SENHA INVÁLIDA
GAMERTAG INEXISTENTE	401 · USUÁRIO NÃO ENCONTRADO

SE O AVATAR NÃO FOR ENCONTRADO, O LOGIN SEGUE SEM ELE.

GAMERTAG ÚNICA

Alice, alice e ALICE são a mesma pessoa

ANTES

```
-- cadastro: diferenciava maiúsculas
WHERE nome_usuario = $1

-- edição: erro cru do Postgres
duplicate key value violates
unique constraint ...
```

Dava para criar "ALICE" com "alice" já existente, e o Perfil mostrava a mensagem técnica do banco.

DEPOIS

```
-- init.sql
CREATE UNIQUE INDEX
  usuario_nome_usuario_lower_idx
  ON usuario (LOWER(nome_usuario));

// userService.js
if (err.code !== '23505') {
  return err;
}
return conflictError(
  'Gamertag já está em uso'); // 409
```

PERFIL E AVATAR

CADA UM ALTERA SÓ OS PRÓPRIOS DADOS

USERROUTES.JS · ANTES

```
router.put('/:id',
  userController.updateUser);
```

USERROUTES.JS · DEPOIS

```
router.put('/:id',
  authenticate, onlyOwner,
  userController.updateUser);
```

O MESMO VALE PARA TROCAR A SENHA E SALVAR O AVATAR.

MIDDLEWARE/AUTH.JS · NOVO

```
function onlyOwner(req, res, next) {
  if (String(req.user.id) !==
      String(req.params.id)) {
    return res.status(403).json({...});
  }
  next();
}
```

SEM TOKEN	401
TOKEN EXPIRADO	401 · FAÇA LOGIN
ID DE OUTRA PESSOA	403

CORREÇÕES DE INTEGRAÇÃO

BUGS ENCONTRADOS NO CAMINHO

PROBLEMA	EFEITO	CORREÇÃO
INIT.SQL SEM VÍRGULA E COM A COLUNA	BANCO NOVO NÃO ERA CRIADO; CADASTRO	VÍRGULA E NOME PADRONIZADO:
SEED GRAVAVA TIPO 'JOGADOR'	USUÁRIOS DE TESTE NÃO JOGAVAM NEM ENTRAVAM NO RANKING	'USUARIO' COM PERFIL 'ALUNO' E AMIZADES DE EXEMPLO
LOGOUT MANTINHA TIPO E PERFIL SALVOS	PRÓXIMO USUÁRIO HERDAVA O PERFIL ANTERIOR	LIMPA TODAS AS CHAVES DA SESSÃO
IMAGEM DO JOGO POR CAMINHO FIXO	XADREZ (.PNG) FICAVA SEM IMAGEM	USA URL_IMAGEM DO BANCO
CADASTRO DEVOLVIA O HASH DA SENHA	DADO SENSÍVEL NA RESPOSTA	RETORNA SÓ CAMPOS SEGUROS

BRANCH FEAT/PARTE-SPRINT3

O QUE MUDOU NO REPOSITÓRIO

17	37	+2.931	−1.138
COMMITS	ARQUIVOS	LINHAS ADICIONADAS	LINHAS REMOVIDAS

ÁREA	ARQUIVOS
FRONT-END · 21	HOME, GAMEDETAILS, LOGIN, PERFIL, AVATAR · NAVBAR, LAYOUT, FOOTER, GAMECARD (NOVO) · GAMESSERVICE, UTILS/PERFIL (NOVO) · INDEX.HTML · DOCKERFILE, NGINX.CONF
BACK-END · 11	USERSERVICE, GAMESSERVICE · USER E AVATAR (CONTROLLERS E ROTAS) · MIDDLEWARE/AUTH (NOVO) · DB.JS · SEED.JS ·
BANCO E DOCKER · 2	INIT.SQL · DOCKER-COMPOSE.YML
TESTES E DOCS · 3	SPRINT3-E2E.SPEC.JS · AVATAR-E2E.SPEC.JS · RELATORIO_SPRINT_03_LUDICA.MD

COMPARAÇÃO COM A BRANCH FIX/BUG-FIXES, SEM CONTAR IMAGENS E VÍDEO

EQUIPE

COMO O GRUPO SE ORGANIZOU

LUCAS

DOCKER DO FRONT E DO BACK, HOME E DETALHE DO JOGO, LOGIN POR E-MAIL, CORREÇÕES DO INIT.SQL E DO SEED

JOÃO VICTOR

PERFIS ALUNO E PROFESSOR, GAMERTAG ÚNICA SEM DIFERENCIAR MAIÚSCULAS, AUTENTICAÇÃO NO PERFIL E NO AVATAR

PATRICIA

CORRIGIU DIVERSOS BUGS, TESTES COM PLAYWRIGHT (FLUXOS DA SPRINT E DO AVATAR), VÍDEO DA EXECUÇÃO, DOCUMENTAÇÃO, RELATÓRIO DAS SPRINTS

FLUXO: UMA BRANCH DA SPRINT (FEAT/PARTE-SPRINT3) A PARTIR DA FIX/BUG-FIXES, UM COMMIT POR FUNCIONALIDADE E PULL REQUEST PARA REVISÃO ANTES DO MERGE.

COMUNICAÇÃO: [COMO O GRUPO COMBINOU AS TAREFAS, EX.: GRUPO DE MENSAGENS E REUNIÕES]

DIFICULDADES

O QUE DEU MAIS TRABALHO

BANCO QUE NÃO SUBIA DO ZERO

O POSTGRES SÓ RODA O INIT.SQL NUM BANCO VAZIO. ERROS DE SINTAXE FICAVAM ESCONDIDOS ATÉ ALGUÉM RECRIAR O VOLUME COM DOWN -V.

NOMES DIFERENTES PARA A MESMA COISA

TIPO_JOGADOR OU PERFIL_USUARIO, 'JOGADOR' OU 'USUARIO': BANCO, SEED, BACK E FRONT PRECISAVAM FALAR A MESMA LÍNGUA.

DOCKER NO WINDOWS

BCRYPT COMPILADO PARA WINDOWS, LOCALHOST QUE MUDA DE SENTIDO DENTRO DO CONTAINER E FINAIS DE LINHA CRLF.

TRÊS PESSOAS, MESMOS ARQUIVOS

USERSERVICE, SEED E INIT.SQL FORAM MEXIDOS POR MAIS DE UM INTEGRANTE; FOI PRECISO TESTAR A INTEGRAÇÃO DEPOIS DE CADA PUSH.

O QUE VEM NAS PRÓXIMAS SPRINTS

SPRINT 4

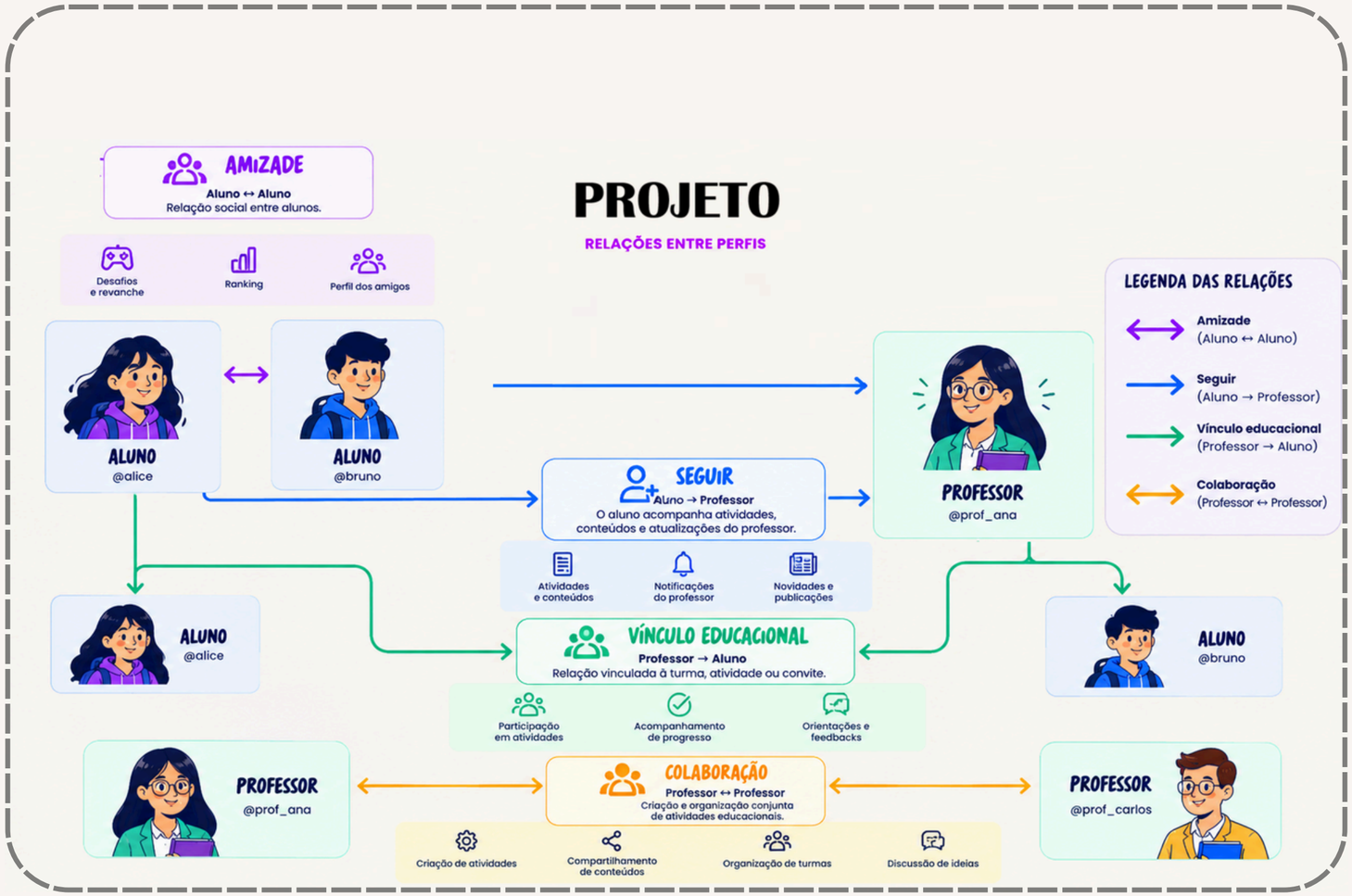
● RELAÇÕES ENTRE PERFIS: AMIZADE, ACOMPANHAMENTO E COLABORAÇÃO

AMIZADE PARA INTERAÇÃO SOCIAL · SEGUIR PARA ACOMPANHAMENTO · COLABORAÇÃO EM ATIVIDADES

- FILTROS DA HOME POR NÍVEL, DISCIPLINA E TEMA
- APLICAR A NOVA IDENTIDADE VISUAL EM AMIGOS, PERFIL, BIBLIOTECA E LOGIN
- INTEGRAR A BRANCH DA SPRINT À VERSÃO PRINCIPAL

MAIS ADIANTE

- AÇÕES EXCLUSIVAS DO ADMINISTRADOR
- JOGOS COMO APLICAÇÕES SEPARADAS, VIA API
- DESAFIAR AMIGO E REVANCHE
- LOJA DE ITENS DO AVATAR UTILIZANDO PONTOS
- SISTEMA DE CONQUISTAS



LÚDICA

OBRIGADO!

PERGUNTAS?

LUCAS ALVES BITTENCOURT · JOÃO VICTOR · PATRICIA SALES MANSANO

github.com/marceloareas/Ludica